Basic C# Interview Questions

1. **What is C#?**
    - C# is a modern, object-oriented, and type-safe programming language developed by Microsoft. It is primarily used for developing Windows applications and web services.
2. **What are the main features of C#?**
    - Object-Oriented
    - Type-Safety
    - Garbage Collection
    - Language Interoperability
    - Exception Handling
    - Multithreading support
3. **What is the difference between == and Equals() in C#?**
    - == is used to compare values of primitive types and reference types (for reference types, it checks if both references point to the same object in memory).
    - Equals() is used to compare the actual content of objects (it can be overridden in custom classes to provide specific comparison logic).
4. **Explain the concept of a namespace in C#.**
    - A namespace is a container that holds classes, structs, interfaces, and other namespaces. It is used to organize code and avoid naming conflicts.
5. **What are the different types of data types in C#?**
    - **Value Types**: int, char, float, double, etc.
    - **Reference Types**: string, array, class, interface, etc.
6. **What is the difference between Array and ArrayList in C#?**
    - **Array**: Fixed size, strongly typed, stores elements of the same type.

- **ArrayList**: Dynamic size, stores elements of any type (not type-safe), part of System.Collections.

7. **What is a constructor in C#?**
    - A constructor is a special method used to initialize objects. It has the same name as the class and no return type.
8. **What is the purpose of the static keyword in C#?**
    - static is used to declare members (variables, methods, etc.) that belong to the class itself, rather than instances of the class.
9. **What is the difference between public, private, and protected access modifiers?**
    - public: The member is accessible from anywhere.
    - private: The member is accessible only within the same class.
    - protected: The member is accessible within the same class or derived classes.
10. **What is a delegate in C#?**
    - A delegate is a type-safe function pointer that holds references to methods with a particular parameter list and return type.

Intermediate C# Interview Questions

1. **Explain the concept of inheritance in C#.**
    - Inheritance allows one class (child class) to inherit fields and methods from another class (base class). It supports code reuse and polymorphism.
2. **What is polymorphism in C#?**
    - Polymorphism allows objects of different types to be treated as objects of a common base type. The two types of polymorphism in C# are method overriding and method overloading.
3. **What are interfaces in C#?**
    - An interface defines a contract for classes or structs to implement. It can contain method declarations but no implementation.

4. **What is an abstract class in C#?**
    - An abstract class cannot be instantiated directly. It can contain abstract methods (without implementation) that must be implemented by derived classes.
5. **What are try, catch, and finally blocks in C#?**
    - These blocks are used for exception handling.
        - try: Contains code that may throw an exception.
        - catch: Catches and handles exceptions.
        - finally: Executes code after the try and catch, regardless of whether an exception occurred.
6. **Explain the difference between string and StringBuilder in C#.**
    - string is immutable, meaning any modification creates a new object. StringBuilder is mutable and is more efficient for operations that involve many changes to a string.
7. **What is a using statement in C#?**
    - The using statement defines a scope for objects that implement IDisposable, ensuring that resources are properly disposed of when the scope ends.
8. **What is the purpose of the LINQ in C#?**
    - LINQ (Language Integrated Query) provides a set of methods for querying and manipulating data from different data sources like arrays, lists, XML, and databases in a consistent manner.
9. **What is the difference between ref and out in C#?**
    - Both are used to pass arguments by reference, but:
        - ref: The variable must be initialized before being passed to the method.
        - out: The variable does not need to be initialized before being passed.
10. **What are extension methods in C#?**

- Extension methods allow you to add functionality to existing types without modifying their source code, by creating static methods in a static class.

Advanced C# Interview Questions

1. **What is the difference between IEnumerable and IEnumerator in C#?**
    - IEnumerable provides a way to iterate over a collection, whereas IEnumerator is the actual iterator that performs the iteration.
2. **What is the purpose of the async and await keywords in C#?**
    - These keywords are used for asynchronous programming. async marks a method as asynchronous, while await is used to pause the method execution until the awaited task is completed.
3. **What is Dependency Injection (DI) in C#?**
    - DI is a design pattern used to achieve Inversion of Control (IoC), where dependencies are injected into a class instead of being created within the class. This improves testability and decouples components.
4. **What are generics in C#?**
    - Generics allow you to define type-safe classes, methods, and collections without knowing the specific type at compile time. It helps in creating reusable code.
5. **Explain Task and Thread in C#.**
    - A Thread is a low-level construct representing an independent path of execution. A Task is a higher-level abstraction that represents an asynchronous operation and is part of the Task Parallel Library (TPL).
6. **What is the difference between Task and Thread in C#?**
    - Task is part of the TPL and is used for parallel programming and asynchronous operations. Thread represents a separate line of execution but lacks the overhead and management features of Task.

7. **What is the difference between readonly and const in C#?**
    - readonly: A field that can only be assigned once, typically in the constructor.
    - const: A constant value that is determined at compile-time and cannot be changed.
8. **What is reflection in C#?**
    - Reflection allows you to inspect and interact with types, properties, methods, and other metadata at runtime. It is often used for tasks like object serialization or ORM frameworks.
9. **What is the garbage collector in C#?**
    - The garbage collector automatically manages memory in C# by freeing objects that are no longer being used, helping prevent memory leaks.
10. **Explain the concept of value types and reference types.**
    - Value types hold the data directly and are stored in the stack (e.g., int, struct). Reference types store references to the data, which is stored in the heap (e.g., class, array).
11. **What is the lock keyword in C#?**
    - lock is used to ensure that only one thread can access a critical section of code at a time, which helps in synchronizing access to shared resources.
12. **What is async/await and what problems do they solve?**
    - async and await are used for writing asynchronous code in a more readable and manageable way. They allow non-blocking I/O operations without using threads explicitly, improving the performance of I/O-bound applications.
13. **Explain Immutable Objects and their importance in C#.**
    - Immutable objects cannot be changed after they are created. In C#, immutability is critical for thread-safety and functional-style programming.

****

1. What is the difference between int and Int32 in C#?

- Both int and Int32 represent the same data type in C#. int is an alias for System.Int32. The difference is that int is more commonly used, while Int32 is part of the .NET Framework's System namespace.

```
int num1 = 5; // Uses 'int'

Int32 num2 = 10; // Uses 'Int32'
```

2. What is a struct in C#?

- A struct is a value type that is used to define lightweight objects that have fields and methods. It’s often used for small data structures like points, rectangles, etc.

```
struct Point

{

   public int X;

   public int Y;

}
```

3. What is the is keyword used for in C#?

- The is keyword is used to check if an object is of a certain type.

```
object obj = "Hello";

if (obj is string)

{

   Console.WriteLine("It's a string!");

}
```

4. What is the difference between throw and throw ex in C#?

- throw is used to rethrow the current exception while maintaining the original stack trace, whereas throw ex rethrows the exception but loses the original stack trace.

```
try

{

   throw new Exception("An error occurred");
```

```
}
catch (Exception ex)
{
    throw; // Rethrow without losing stack trace
}
```

5. What is the checked keyword used for in C#?

- The checked keyword is used to explicitly enable overflow checking for arithmetic operations.

```
int result = checked(2147483647 + 1); // Will throw an OverflowException
```

6. What is the difference between string and StringBuilder?

- string is immutable, meaning that once created, it cannot be modified. StringBuilder is mutable and is more efficient when performing multiple string modifications.

```
string s = "Hello";
s += " World"; // Creates a new string


StringBuilder sb = new StringBuilder("Hello");
sb.Append(" World"); // Modifies the existing string
```

7. How would you handle a situation where you need to allow for multiple exceptions in one try block?

- You can use multiple catch blocks to handle different exceptions.

```
try
{
    // Code that may throw exceptions
}
catch (ArgumentNullException ex)
{
    Console.WriteLine(ex.Message);
```

```
}
catch (FormatException ex)
{
   Console.WriteLine(ex.Message);
}
```

8. Explain what a null reference is in C#.

- A null reference occurs when you try to access an object that is not initialized (i.e., it is set to null).

```
string str = null;
Console.WriteLine(str.Length); // Throws NullReferenceException
```

9. How do you handle null values in C#?

- You can handle null values by using null checks (== null) or null-coalescing operator (??).

```
string str = null;
Console.WriteLine(str?.Length); // Safe call that returns null if str is null


int? num = null;
Console.WriteLine(num ?? 0); // Returns 0 if num is null
```

10. What is the purpose of the base keyword?

- The base keyword is used to access members of a base class from a derived class.

```
class Animal
{
   public void Speak() => Console.WriteLine("Animal speaks");
}


class Dog : Animal
{
```

```
    public void Speak() => base.Speak(); // Calls the base class Speak method
}
```

11. What does the readonly keyword do?

- The readonly keyword ensures that a field can only be assigned a value once, either in its declaration or in the constructor.

```
class MyClass
{
    public readonly int myValue = 10;
}
```

12. What is the purpose of the new keyword in method hiding in C#?

- The new keyword is used to hide a member of a base class with a member of the same name in a derived class.

```
class BaseClass
{
    public void Display() => Console.WriteLine("Base class");
}


class DerivedClass : BaseClass
{
    public new void Display() => Console.WriteLine("Derived class");
}
```

13. How can you force a method to execute after all the objects have been disposed of in C#?

- You can use the Dispose method in a class that implements the IDisposable interface to clean up resources after an object is no longer in use.

```
class MyClass : IDisposable
{
```

```
    public void Dispose()
    {
        // Cleanup resources
    }
}
```

14. What are access modifiers in C#?

- Access modifiers in C# control the visibility and accessibility of members of a class. They include public, private, protected, and internal.

```
public class MyClass
{
    private int privateField;
    public int publicField;
}
```

15. Explain the difference between const and readonly in C#?

- const is for compile-time constant values, whereas readonly is for run-time constant values that can only be set in a constructor or field initializer.

```
const int x = 10; // Compile-time constant
readonly int y = 20; // Can be set in the constructor
```

16. What is boxing and unboxing in C#?

- **Boxing** is the process of converting a value type to a reference type, while **unboxing** is the reverse operation.

```
int num = 10;
object obj = num; // Boxing


int unboxedNum = (int)obj; // Unboxing
```

17. What is the difference between == and ReferenceEquals() in C#?

- == checks if two objects are equal, depending on the type's Equals() method, while ReferenceEquals() checks if two references point to the same object.

```
object obj1 = new object();

object obj2 = new object();

Console.WriteLine(obj1 == obj2); // False

Console.WriteLine(ReferenceEquals(obj1, obj2)); // False
```

18. What is an anonymous method in C#?

- An anonymous method is a method without a name, defined inline.

```
Func<int, int, int> add = delegate(int a, int b) { return a + b; };

Console.WriteLine(add(5, 10)); // Outputs 15
```

19. What is the difference between List<T> and ArrayList in C#?

- List<T> is a generic collection, which provides type safety and performance benefits, while ArrayList is non-generic and stores elements as objects.

```
List<int> list = new List<int> { 1, 2, 3 };

ArrayList arrayList = new ArrayList { 1, "String", 3.5 };
```

20. What are checked and unchecked keywords in C#?

- checked forces arithmetic overflow checking, while unchecked disables it.

```
int result = checked(2147483647 + 1); // OverflowException

int result2 = unchecked(2147483647 + 1); // No exception
```

These are just some of the topics. Let me know if you'd like more detailed explanations or examples for the remaining questions!

Sure! Here's the continuation with detailed explanations and examples for the remaining questions:

21. What is the difference between Array and ArrayList in C#?

- **Array** is a fixed-size collection of elements, all of the same type, whereas **ArrayList** is a non-generic collection that can store elements of any type (objects). Array provides better performance, as its size is fixed and types are known at compile time.

```
int[] intArray = new int[] { 1, 2, 3 };  // Fixed size and type

ArrayList arrayList = new ArrayList();   // Can hold any type

arrayList.Add(1);

arrayList.Add("string");
```

22. What is the difference between Task and Thread in C#?

- A **Thread** represents an independent path of execution in a program, while a **Task** is an abstraction over threads, providing more control over asynchronous operations. Task is part of the Task Parallel Library (TPL) and is used for parallel and asynchronous operations.

```
// Thread example

Thread thread = new Thread(() => { Console.WriteLine("Running in a thread"); });

thread.Start();


// Task example

Task.Run(() => { Console.WriteLine("Running in a task"); });
```

23. What is a namespace in C# and why is it used?

- A **namespace** is a container for classes, structs, and other types to organize code logically. It helps avoid name conflicts and provides a way to group related classes.

```
namespace MyApp

{

   class MyClass

   {

      public void Display() => Console.WriteLine("Hello from MyClass");

   }
```

}

24. What is the difference between IEnumerable and ICollection in C#?

- **IEnumerable** is the base interface for all collections that can be enumerated, whereas **ICollection** extends IEnumerable and adds methods for adding, removing, and checking the size of the collection.

IEnumerable<int> enumerable = new List<int> { 1, 2, 3 }; // Only iteration

ICollection<int> collection = new List<int> { 1, 2, 3 }; // Can add/remove items

25. What is the default keyword in C#?

- The default keyword is used to return the default value for a type. For value types, it returns the zeroed value (e.g., 0 for integers, false for booleans), and for reference types, it returns null.

int x = default(int); // 0

string str = default(string); // null

26. What is the difference between == and Equals() method in C#?

- The == operator checks for reference equality for reference types and value equality for value types. The Equals() method is meant to check for logical equality, and can be overridden in custom classes to provide deep comparisons.

string str1 = "hello";

string str2 = "hello";

Console.WriteLine(str1 == str2); // True

Console.WriteLine(str1.Equals(str2)); // True

27. How does try-catch-finally block work in C#?

- The try block contains code that might throw an exception. If an exception occurs, the catch block handles it. The finally block, if present, always executes, regardless of whether an exception was thrown.

try

{

```
    int result = 10 / 0;  // Throws DivideByZeroException
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("Error: " + ex.Message);
}
finally
{
    Console.WriteLine("This block is always executed.");
}
```

28. What is the difference between ref and out parameters in C#?

- Both ref and out are used to pass parameters by reference, but with ref, the parameter must be initialized before it is passed, whereas with out, the parameter doesn't need to be initialized, and it must be assigned a value before the method exits.

```
void RefExample(ref int x) { x += 10; }
void OutExample(out int x) { x = 10; }
int a = 5;
RefExample(ref a);  // a = 15
int b;
OutExample(out b);  // b = 10
```

29. What are extension methods in C#?

- Extension methods allow you to "add" new methods to existing types without modifying their source code. This is done by creating static methods in a static class, and the first parameter of the method specifies the type to extend.

```
public static class StringExtensions
{
    public static string Reverse(this string str)
```

```
    {
        return new string(str.Reverse().ToArray());
    }
}
```

```
string str = "hello";
Console.WriteLine(str.Reverse());  // Outputs "olleh"
```

30. Explain the concept of garbage collection in C#.

- Garbage collection in C# is the automatic process of reclaiming memory that is no longer in use. The garbage collector (GC) runs periodically to remove objects that are no longer referenced.

```
class MyClass
{
    public void Dispose() { /* Cleanup code */ }
}

MyClass obj = new MyClass();
obj = null;  // The object is now eligible for garbage collection
```

31. What is a Tuple in C# and how is it used?

- A Tuple is a data structure that can hold a fixed number of items of different types. It is useful when you need to return multiple values from a method.

```
Tuple<int, string> GetPerson()
{
    return new Tuple<int, string>(1, "John");
}

var person = GetPerson();
```

```
Console.WriteLine(person.Item1);  // 1

Console.WriteLine(person.Item2);  // John
```

32. What is the difference between a deep copy and a shallow copy in C#?

- A **shallow copy** copies the references of objects, so if the original object is modified, the copied object reflects those changes. A **deep copy** creates a completely new copy of the original object and all objects contained within it.

```
// Shallow copy

MyClass original = new MyClass { Name = "Test" };

MyClass shallowCopy = original; // Both point to the same object


// Deep copy (implement custom logic or use serialization)

MyClass deepCopy = new MyClass { Name = original.Name };
```

33. Explain how to implement inheritance in C#?

- Inheritance allows a class to derive from another class, inheriting its members (fields, methods). It provides a way to reuse code and create a hierarchical class structure.

```
class Animal

{

   public void Eat() => Console.WriteLine("Eating...");

}


class Dog : Animal

{

   public void Bark() => Console.WriteLine("Barking...");

}


Dog dog = new Dog();
```

```
dog.Eat();  // Inherited method

dog.Bark(); // Derived method
```

34. What is method overloading and method overriding in C#?

- **Method overloading** occurs when multiple methods with the same name exist but with different parameters. **Method overriding** occurs when a derived class provides a new implementation for a method defined in a base class.

```
// Overloading

void Print(int i) => Console.WriteLine(i);

void Print(string s) => Console.WriteLine(s);


// Overriding

class BaseClass

{

   public virtual void Display() => Console.WriteLine("Base");

}


class DerivedClass : BaseClass

{

   public override void Display() => Console.WriteLine("Derived");

}
```

35. How does async and await work in C#?

- async and await are used for asynchronous programming. async modifies a method to be asynchronous, and await pauses the method's execution until the awaited task is complete.

```
async Task<string> GetDataAsync()

{

   await Task.Delay(2000); // Simulate a delay
```

```
    return "Data received!";
}

string result = await GetDataAsync();
Console.WriteLine(result);  // Outputs after 2 seconds
```

36. What is the difference between an abstract class and an interface in C#?

- An **abstract class** can contain both abstract (unimplemented) methods and regular methods with implementations. An **interface** only defines method signatures and properties, but no implementation.

```
abstract class Animal
{
    public abstract void Speak();
    public void Sleep() => Console.WriteLine("Sleeping...");
}

interface IAnimal
{
    void Speak();
}
```

Here’s the continuation of the remaining explanations and examples:

37. What are events and delegates in C#?

- **Delegates** are type-safe function pointers that allow you to reference methods with a specific signature. **Events** are a way to provide notifications to clients when something of interest happens in an object. Events are usually associated with delegates.

```
public delegate void Notify(); // Delegate
```

```
public class Process
{
    public event Notify ProcessCompleted; // Event

    public void StartProcess()
    {
        // Perform process tasks
        Console.WriteLine("Process Started...");
        ProcessCompleted?.Invoke(); // Trigger the event
    }
}

Process process = new Process();
process.ProcessCompleted += () => Console.WriteLine("Process Completed!");
process.StartProcess();
```

38. What are properties in C#?

- **Properties** are members of a class that provide a mechanism to read, write, or compute the values of private fields. They act as a combination of methods and fields, with get and set accessors.

```
public class Person
{
    private string name;
    public string Name
    {
        get { return name; }
        set { name = value; }
    }
```

```
}
```

39. What is the difference between public, private, protected, and internal access modifiers in C#?

- **public** allows access from anywhere.
- **private** restricts access to within the same class.
- **protected** allows access within the same class or derived classes.
- **internal** restricts access to within the same assembly.

```
public class MyClass
{
    public int PublicField;
    private int PrivateField;
    protected int ProtectedField;
    internal int InternalField;
}
```

40. What is a sealed class in C#?

- A **sealed class** cannot be inherited. This is useful when you want to prevent a class from being subclassed.

```
public sealed class SealedClass
{
    public void Display() => Console.WriteLine("Sealed Class");
}


// This will throw a compile-time error
// public class DerivedClass : SealedClass { }
```

41. Explain the concept of multithreading in C#.

- **Multithreading** in C# is the ability to run multiple threads (independent units of execution) concurrently. This can improve

performance in applications that perform complex calculations or I/O operations.

```
Thread thread = new Thread(() => { Console.WriteLine("Running in a separate thread"); });

thread.Start();
```

42. What is the difference between String and StringBuilder in C#?

- **String** is immutable, meaning any operation on a string creates a new string. **StringBuilder** is mutable, meaning it allows modifications without creating new objects, making it more efficient for repeated string manipulation.

```
// Using String (Inefficient)

string str = "Hello";

str += " World"; // Creates a new string object


// Using StringBuilder (Efficient)

StringBuilder sb = new StringBuilder("Hello");

sb.Append(" World"); // Modifies the existing object
```

43. What are Value Types and Reference Types in C#?

- **Value types** hold the actual data (e.g., int, double, struct). When a value type is assigned to a new variable, a copy of the data is created.
- **Reference types** store references to the data (e.g., string, class, array). When a reference type is assigned to a new variable, both variables point to the same data.

```
int a = 5; // Value type

string str = "hello"; // Reference type
```

44. What is an interface in C# and when would you use it?

- An **interface** defines a contract for classes to follow, specifying methods, properties, and events that must be implemented. It is used to define a common behavior that different classes can implement.

```
interface IShape
```

```
{
   void Draw();
}


class Circle : IShape
{
   public void Draw() => Console.WriteLine("Drawing Circle");
}
```

45. How is the lock statement used in C# for synchronization?

- The **lock** statement is used to ensure that a block of code is executed by only one thread at a time, preventing race conditions when accessing shared resources.

```
private static object _lock = new object();


public void SomeMethod()
{
   lock (_lock)
   {
      // Critical code block that should be executed by one thread at a time
   }
}
```

46. What is the use of using directive in C#?

- The **using** directive is used to include namespaces so that the types in those namespaces can be used without fully qualifying their names. It is also used to define a scope for resources that need to be disposed.

```
using System;
using System.IO;
```

```
FileStream fs = new FileStream("example.txt", FileMode.Open);
```

47. What is yield return in C#?

- The **yield return** statement is used to return each element one at a time from an iterator method, enabling the method to be used in a foreach loop. It allows deferred execution.

```
public IEnumerable<int> GetNumbers()
{
   yield return 1;
   yield return 2;
   yield return 3;
}


foreach (var number in GetNumbers())
{
   Console.WriteLine(number);
}
```

48. What are static classes in C#?

- A **static class** cannot be instantiated and can only contain static members. It is used to group related methods that do not require object state.

```
public static class MathHelper
{
   public static int Add(int a, int b) => a + b;
}
```

49. What is the dynamic keyword in C#?

- The **dynamic** keyword is used to declare a variable that bypasses compile-time type checking. It is resolved at runtime, allowing you to work with objects of unknown types.

```
dynamic value = 10;
```

```
value = "Hello";  // No compile-time errors

Console.WriteLine(value);
```

50. What is Dependency Injection (DI) in C# and how is it implemented?

- **Dependency Injection** is a design pattern where an object's dependencies are provided by an external source, rather than the object creating them. It promotes loose coupling and easier unit testing.

```
public interface ICar
{
   void Drive();
}


public class Car : ICar
{
   public void Drive() => Console.WriteLine("Driving a car");
}


public class Driver
{
   private readonly ICar _car;


   public Driver(ICar car) => _car = car;


   public void DriveCar() => _car.Drive();
}
```

// In a DI container (e.g., in a framework like ASP.NET Core), you inject dependencies

Here are the remaining answers for the C# interview questions:

51. Explain the Task Parallel Library (TPL) and its usage in C#?

- The **Task Parallel Library (TPL)** in C# simplifies parallel programming by providing the Task class to represent asynchronous operations. It allows you to execute code concurrently and manage multiple tasks in a more efficient manner.

```
Task.Run(() => {
    // Code to run asynchronously
    Console.WriteLine("Task running");
});
```

52. What is the difference between await and Task.Wait() in C#?

- **await** is used with asynchronous methods and allows the program to continue executing other tasks while waiting for the task to complete. **Task.Wait()** blocks the thread until the task completes, which can make the application less responsive.

```
// Await allows asynchronous waiting
await SomeAsyncMethod();


// Wait blocks the current thread
task.Wait();
```

53. How can you implement a custom exception class in C#?

- You can create a custom exception by inheriting from the Exception class and adding any additional properties or methods as needed.

```
public class CustomException : Exception
{
    public CustomException(string message) : base(message) { }
}
```

```
throw new CustomException("Something went wrong");
```

54. What is reflection in C# and when is it used?

- **Reflection** allows you to inspect and interact with types, methods, properties, and attributes at runtime. It is commonly used in scenarios like serialization, object mapping, or creating flexible applications.

```
var type = typeof(MyClass);

var methods = type.GetMethods(); // Get all methods in MyClass
```

55. What are attributes in C# and how are they used?

- **Attributes** are metadata that provide additional information about types, methods, properties, etc. They are used to control behaviors, validation, or documentation.

```
[Obsolete("This method is deprecated")]

public void OldMethod() { }
```

56. What is the difference between IDisposable and a finalizer in C#?

- **IDisposable** is an interface used to implement the Dispose method, which is called to release unmanaged resources explicitly. A **finalizer** (destructor) is called automatically when an object is garbage collected to clean up unmanaged resources.

```
// IDisposable

public class Resource : IDisposable

{

   public void Dispose() { /* Release unmanaged resources */ }

}


// Finalizer

~Resource() { /* Cleanup code */ }
```

57. What is the async method in C# and when would you use it?

- An **async** method is used to indicate that the method contains asynchronous operations, making it easier to work with I/O-bound operations. It allows the method to return a Task and can use await to call other async methods.

```
public async Task<string> FetchDataAsync()
{
   var result = await SomeApiCallAsync();
   return result;
}
```

58. What is Polymorphism in C# and how is it implemented?

- **Polymorphism** is the ability of different objects to respond to the same method call in different ways. It is implemented via method overriding (runtime polymorphism) or method overloading (compile-time polymorphism).

```
// Runtime Polymorphism
public class Animal
{
   public virtual void Speak() => Console.WriteLine("Animal speaks");
}

public class Dog : Animal
{
   public override void Speak() => Console.WriteLine("Bark");
}

Animal myAnimal = new Dog();
myAnimal.Speak(); // Outputs: Bark
```

59. What is the difference between ArrayList and List<T> in C#?

- **ArrayList** is a non-generic collection that can store any type of objects, but it can be slower and type-unsafe. **List<T>** is a generic collection, type-safe, and offers better performance by using a specific data type.

```
ArrayList arrayList = new ArrayList();

arrayList.Add(10); // Can store any type


List<int> list = new List<int>();

list.Add(10); // Type-safe
```

60. Explain how Nullable types work in C#?

- **Nullable types** allow value types to represent null, which is useful for cases where you need to represent the absence of a value, especially with databases.

```
int? nullableInt = null;

if (nullableInt.HasValue)

{

   Console.WriteLine(nullableInt.Value);

}
```

61. What is Method Chaining in C# and how does it work?

- **Method chaining** is a technique where multiple methods are called on the same object in a single statement. Each method returns the object itself (this), allowing successive method calls.

```
public class StringBuilderExample

{

   public StringBuilderExample AppendText(string text)

   {

      // Append logic

      return this; // Return current instance for chaining

   }
```

```
}
```

```
new StringBuilderExample().AppendText("Hello").AppendText(" World");
```

62. What is Tuple and ValueTuple in C#?

- **Tuple** is a class that can store a fixed number of elements of different types. **ValueTuple** is a lightweight, value-type alternative to Tuple, introduced in C# 7.0, which is more efficient.

```
// Tuple (reference type)

var tuple = new Tuple<int, string>(1, "Apple");


// ValueTuple (value type)

(int, string) valueTuple = (1, "Apple");
```

63. Explain the concept of Reflection in C#?

- **Reflection** allows you to inspect the metadata of assemblies, types, and members. It is used to dynamically invoke methods or access properties at runtime.

```
var type = typeof(MyClass);

var methods = type.GetMethods();

foreach (var method in methods)

{

   Console.WriteLine(method.Name);

}
```

64. What is the volatile keyword in C#?

- The **volatile** keyword indicates that a field can be accessed by multiple threads. It ensures that the value of the field is always read from memory and not cached by the thread.

```
private volatile bool flag = false;
```

65. What are async methods and how do they help with concurrency in C#?

- **Async methods** are designed to run asynchronously, making it easier to handle long-running I/O operations without blocking the main thread. They allow for better responsiveness and scalability in applications.

```
public async Task<string> GetDataAsync()
{
    var data = await DownloadDataFromWebAsync();
    return data;
}
```

66. How does LINQ (Language Integrated Query) work in C#?

- **LINQ** allows querying of collections (like arrays, lists, etc.) using SQL-like syntax directly in C#. It provides a more readable and concise way of filtering, sorting, and transforming data.

```
var numbers = new List<int> { 1, 2, 3, 4, 5 };
var evenNumbers = from n in numbers
                  where n % 2 == 0
                  select n;
```

67. What is ImmutableObject and how would you implement it in C#?

- An **immutable object** is one whose state cannot be changed once it is created. In C#, you can implement this by making fields readonly and not providing any setters.

```
public class ImmutablePerson
{
    public string Name { get; }
    public int Age { get; }

    public ImmutablePerson(string name, int age)
    {
```

```
        Name = name;
        Age = age;
    }
}
```

68. What is Exception Handling in C# and how does it work?

- **Exception handling** in C# is done using try, catch, and finally blocks. The try block contains code that might throw an exception, the catch block handles exceptions, and the finally block runs code that must execute regardless of exceptions.

```
try
{
    int result = 10 / 0;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Cleanup code here");
}
```

69. What is the Dynamic Language Runtime (DLR) in C#?

- The **Dynamic Language Runtime (DLR)** is a set of services in .NET that enable dynamic typing, dynamic method invocation, and other features typically found in dynamic languages. It is used to support languages like Python and Ruby on the .NET platform.

70. What is ThreadPool in C# and when should it be used?

- The **ThreadPool** provides a pool of worker threads that can be used to execute tasks concurrently. It is efficient for short-lived tasks and helps to reduce the overhead of creating new threads.

```
ThreadPool.QueueUserWorkItem(state => {
    Console.WriteLine("Task running on thread pool");
});
```

71. Explain the concept of Interlocked in C#?

- **Interlocked** is a class that provides atomic operations for variables shared by multiple threads. It is used to ensure thread safety when modifying shared data.

```
int counter = 0;
Interlocked.Increment(ref counter); // Atomically increments the counter
```

72. What is unsafe code in C# and why would you use it?

- **Unsafe code** allows pointer manipulation and direct memory access. It is typically used in performance-critical applications or when working with low-level APIs.

```
unsafe
{
    int* ptr = &someVariable;
    Console.WriteLine(*ptr);
}
```

73. What is the Dispose pattern and how does it work?

- The **Dispose pattern** is used to release unmanaged resources in a class when it is no longer needed. It is typically implemented by the IDisposable interface and can be used in conjunction with a finalizer.

```
public class MyClass : IDisposable
{
    private bool disposed = false;
```

```
    public void Dispose()
    {
        if (!disposed)
        {
            // Release unmanaged resources
            disposed = true;
        }
    }
}
```

74. What is multicast delegation in C#?

- **Multicast delegates** allow a single delegate to call multiple methods. It is useful when you want to notify multiple subscribers or execute multiple methods in response to a single event.

```
public delegate void Notify();


Notify notifyDelegate = Method1;
notifyDelegate += Method2;  // Multicast delegate


notifyDelegate();
```

75. What is the EventHandler and how do you use it in C#?

- The **EventHandler** is a predefined delegate type used to handle events in C#. It is commonly used with events that have no return values.

```
public class MyClass
{
    public event EventHandler MyEvent;


    protected virtual void OnMyEvent()
```

```
    {
        MyEvent?.Invoke(this, EventArgs.Empty);
    }
}
```

76. How does async/await improve the performance of I/O bound tasks in C#?

- **Async/await** allows I/O-bound tasks, such as file or network operations, to be executed asynchronously. This improves performance by freeing up the thread to perform other operations instead of waiting for the I/O operation to complete.

77. What is Tuple in C# and how is it used?

- A **Tuple** is a data structure that can store multiple values of different types. It is used for grouping multiple values into a single object.

```
var tuple = new Tuple<int, string>(1, "apple");
```

78. What is the yield keyword in C# and when would you use it?

- The **yield** keyword is used to return elements one at a time from an iterator method. It allows deferred execution of the method.

```
public IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
}
```

79. What is lambda expression in C#?

- A **lambda expression** is an anonymous method used to create delegates or expression tree types. It is concise and often used with LINQ.

```
Func<int, int, int> add = (x, y) => x + y;
```

80. What are extension methods in C#?

- **Extension methods** allow you to add new functionality to existing types without modifying their source code. They are static methods but are called as if they were instance methods.

```
public static class StringExtensions
{
    public static bool IsPalindrome(this string str)
    {
        return str == new string(str.Reverse().ToArray());
    }
}


string word = "madam";
bool result = word.IsPalindrome();
```

81. What is Dependency Injection and how does it work in C#?

- **Dependency Injection (DI)** is a design pattern used to implement Inversion of Control (IoC) by passing dependencies (objects) to a class rather than having it create them. It improves maintainability and testability.

```
// Constructor Injection
public class MyClass
{
    private readonly IService _service;
    public MyClass(IService service)
    {
        _service = service;
    }
}
```

82. What is Serialization and Deserialization in C#?

- **Serialization** is the process of converting an object into a format that can be easily stored or transmitted (e.g., JSON, XML). **Deserialization** is the reverse process, converting the serialized format back into an object.

```
// Serialization

var json = JsonConvert.SerializeObject(myObject);


// Deserialization

var myObject = JsonConvert.DeserializeObject<MyClass>(json);
```

83. What is IEnumerable and IEnumerator in C#?

- **IEnumerable** is an interface that defines a method GetEnumerator() to return an enumerator that can iterate through a collection. **IEnumerator** is the interface that defines methods MoveNext(), Current, and Reset() to enumerate through the collection.

```
IEnumerable<int> numbers = new List<int> { 1, 2, 3 };

IEnumerator<int> enumerator = numbers.GetEnumerator();
```

84. What is LINQ and how does it differ from SQL?

- **LINQ (Language Integrated Query)** allows querying collections directly in C# using a SQL-like syntax. Unlike SQL, which queries databases, LINQ queries work on in-memory collections such as arrays and lists.

```
var query = from n in numbers

            where n > 2

            select n;
```

85. How can you handle exceptions in C# without using a try-catch block?

- Exceptions can be handled using **using** statements for resource management or by employing **custom error handling** logic without a try-catch block in certain cases like logging or returning default values.

Here are the remaining answers for your C# interview questions:

86. What is the Observer design pattern in C#?

- The **Observer design pattern** defines a one-to-many dependency between objects, so when one object changes state, all its dependents (observers) are notified and updated automatically. This pattern is often used in event handling.

```
// Observer pattern example

public class Subject

{

   private List<IObserver> observers = new List<IObserver>();


   public void Attach(IObserver observer) => observers.Add(observer);

   public void Detach(IObserver observer) => observers.Remove(observer);

   public void Notify() => observers.ForEach(observer =>
observer.Update());

}


public interface IObserver

{

   void Update();

}
```

87. What is the Singleton design pattern in C#?

- The **Singleton pattern** ensures that a class has only one instance and provides a global access point to that instance. It's often used for shared resources like database connections or configuration settings.

```
public class Singleton

{

   private static Singleton instance;


   private Singleton() { }
```

```
    public static Singleton Instance
    {
        get
        {
            if (instance == null)
            {
                instance = new Singleton();
            }
            return instance;
        }
    }
}
```

88. What are weak references in C#?

- A **weak reference** allows you to hold a reference to an object without preventing it from being garbage-collected. It is useful when you need to maintain references to objects, but you don’t want them to prevent garbage collection.

```
WeakReference weakRef = new WeakReference(myObject);
```

89. What is garbage collection and how does it work in C#?

- **Garbage Collection (GC)** is the automatic process by which the .NET runtime reclaims memory used by objects that are no longer referenced. It frees developers from manual memory management. The GC runs periodically and can be triggered by calling GC.Collect() explicitly.

```
// Explicit GC invocation (not generally recommended)

GC.Collect();
```

90. How do you avoid a memory leak in C#?

- To avoid memory leaks in C#, ensure that you properly dispose of unmanaged resources, release event handlers, and avoid holding

references to objects that are no longer needed. Implementing IDisposable and using using blocks can help manage resources.

```
using (var resource = new Resource())
{
   // Use resource
}
```

91. Explain the concept of static classes in C#?

- A **static class** in C# is a class that cannot be instantiated. It only contains static members (fields, methods, properties). Static classes are often used for utility functions, constants, or methods that operate globally.

```
public static class MathUtility
{
   public static int Add(int a, int b) => a + b;
}
```

92. How do you optimize performance in C# applications?

- Performance optimization in C# can involve several techniques:
    - Minimize memory allocations (e.g., use StringBuilder instead of string concatenation).
    - Use value types (structs) when dealing with small data structures.
    - Avoid unnecessary boxing and unboxing.
    - Optimize loops and reduce complexity.
    - Use caching techniques and avoid frequent database calls.

93. What is the IEnumerable<T> interface in C# and how does it work?

- **IEnumerable<T>** is a generic interface that defines a method GetEnumerator() to allow iteration over a collection. It is the base interface for all collections that support iteration (e.g., lists, arrays).

```
IEnumerable<int> numbers = new List<int> { 1, 2, 3 };
```

```
foreach (var number in numbers)
{
    Console.WriteLine(number);
}
```

94. How do you implement and use async methods in C#?

- **Async methods** are implemented using the async keyword. These methods typically return a Task or Task<T>. Inside an async method, you use the await keyword to call other async methods, enabling asynchronous execution.

```
public async Task<int> GetDataAsync()
{
    var data = await SomeAsyncMethod();
    return data;
}
```

95. What is Reflection and what are the use cases for it in C#?

- **Reflection** allows you to inspect and interact with the metadata of types, methods, properties, and other elements in a program at runtime. It is useful for scenarios like dynamic method invocation, serialization, object mapping, and code generation.

```
Type type = typeof(MyClass);
MethodInfo method = type.GetMethod("MyMethod");
method.Invoke(new MyClass(), null); // Invoking method dynamically
```

96. What is yield return in C#?

- The **yield return** keyword is used in an iterator method to return elements one at a time. It helps in deferred execution and is useful for implementing custom collections or enumerators.

```
public IEnumerable<int> GetNumbers()
{
    yield return 1;
```

```
    yield return 2;
    yield return 3;
}
```

97. What is the difference between abstract class and interface?

- An **abstract class** can provide implementation for some methods and define others as abstract. A **interface** only defines method signatures without implementation. A class can implement multiple interfaces but can inherit from only one abstract class.

```
// Abstract class
public abstract class Animal
{
    public abstract void Speak();
}


// Interface
public interface IAnimal
{
    void Speak();
}
```

98. Explain the concept of lazy initialization in C#?

- **Lazy initialization** is a design pattern where an object's initialization is deferred until it is actually needed. In C#, you can use the Lazy<T> class to implement lazy loading.

```
Lazy<MyClass> myClass = new Lazy<MyClass>(() => new MyClass());
MyClass obj = myClass.Value; // Initializes only when needed
```

99. What is Dependency Injection and how does it work in C#?

- **Dependency Injection (DI)** is a design pattern used to implement Inversion of Control (IoC), where objects are passed their dependencies

(services) rather than creating them internally. This pattern helps to make code more modular and testable.

```
// Constructor Injection
public class Car
{
    private readonly IEngine _engine;

    public Car(IEngine engine)
    {
        _engine = engine;
    }
}

// Usage
IEngine engine = new Engine();
var car = new Car(engine);
```

100. What is Serialization and Deserialization in C#?

- **Serialization** is the process of converting an object into a format (e.g., JSON, XML) that can be easily stored or transmitted. **Deserialization** is the process of converting the serialized data back into an object.

```
// Serialize
var json = JsonConvert.SerializeObject(myObject);

// Deserialize
var myObject = JsonConvert.DeserializeObject<MyClass>(json);
```

Here are the answers to the remaining questions:

101. What is IEnumerable and IEnumerator in C#?

- **IEnumerable** is an interface that defines a method GetEnumerator(), which returns an IEnumerator that can iterate through a collection. **IEnumerator** is used to traverse a collection (e.g., through MoveNext() and Current properties).

```
public class MyCollection : IEnumerable<int>
{
   private int[] numbers = { 1, 2, 3 };

   public IEnumerator<int> GetEnumerator()
   {
      foreach (var num in numbers)
         yield return num;
   }

   IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}
```

102. What is LINQ and how does it differ from SQL?

- **LINQ (Language Integrated Query)** is a feature in C# that allows you to write SQL-like queries directly in C# code against collections, arrays, or databases. LINQ queries are strongly typed, while SQL is a database query language.

```
var result = from num in numbers
             where num > 5
             select num;
```

Difference: SQL works on a database (external data source), while LINQ works on in-memory collections or databases via LINQ to SQL or Entity Framework.

103. How can you handle exceptions in C# without using a try-catch block?

- You can handle exceptions by using **Event Handling** or **Global Error Handlers** (like Application.ThreadException or AppDomain.CurrentDomain.UnhandledException for Windows Forms or Console apps).
- Additionally, you can handle errors in asynchronous tasks using Task.ContinueWith().

```
Task.Run(() => { /* code */ })
    .ContinueWith(t => { /* handle exceptions */ },
TaskContinuationOptions.OnlyOnFaulted);
```

104. What is the Observer design pattern in C#?

- **Observer pattern** is a design pattern where an object (subject) maintains a list of dependents (observers) that need to be notified of any changes. It is commonly used in event-driven systems.

```
public class Subject
{
    private List<IObserver> observers = new List<IObserver>();

    public void Attach(IObserver observer) => observers.Add(observer);
    public void Detach(IObserver observer) => observers.Remove(observer);
    public void Notify() => observers.ForEach(observer =>
observer.Update());
}

public interface IObserver
{
    void Update();
}
```

105. What is the Singleton design pattern in C#?

- **Singleton pattern** ensures that a class has only one instance and provides a global point of access to that instance. It's used when you want to control the instantiation of a class.

```
public class Singleton
{
   private static Singleton instance;

   private Singleton() { }

   public static Singleton Instance
   {
      get
      {
         if (instance == null)
         {
            instance = new Singleton();
         }
         return instance;
      }
   }
}
```

106. What are weak references in C#?

- **Weak references** allow an object to be referenced without preventing its garbage collection. They are useful when you need to cache objects but don't want to keep them alive indefinitely.

```
WeakReference weakRef = new WeakReference(someObject);
```

107. What is garbage collection and how does it work in C#?

- **Garbage collection (GC)** is the process by which the .NET runtime automatically reclaims memory from objects that are no longer in use. The GC periodically identifies objects that are unreachable and frees their memory.
- It helps manage memory efficiently, eliminating the need for manual memory management.

```
// Triggering GC manually
GC.Collect();
```

108. How do you avoid a memory leak in C#?

- To avoid memory leaks in C#, ensure proper memory management, including:
    - **Disposing unmanaged resources** using the IDisposable interface.
    - **Unsubscribing from events** to prevent objects from being held in memory unintentionally.
    - Use using blocks for objects that implement IDisposable.
- using (var stream = new FileStream("file.txt", FileMode.Open))
- {
- // Use the stream
- }

109. Explain the concept of static classes in C#?

- A **static class** in C# cannot be instantiated and can only contain static members (methods, properties). It is used when you need a class to hold only global methods or constants.

```
public static class MathUtility
{
   public static int Add(int a, int b) => a + b;
}
```

110. How do you optimize performance in C# applications?

- **Performance optimization** in C# can involve:

- Minimizing memory allocations.
- Using value types (structs) for small data objects.
- Avoiding unnecessary boxing/unboxing.
- Using StringBuilder for string concatenation instead of the + operator.
- Profiling the application to identify performance bottlenecks.
- Using multi-threading or parallel execution when necessary.

111. What is the IEnumerable<T> interface in C# and how does it work?

- **IEnumerable<T>** is a generic interface that allows a collection to be enumerated. It defines a method GetEnumerator() that returns an IEnumerator<T>, which allows for iteration over the collection.

```
public class MyCollection : IEnumerable<int>
{
    private int[] numbers = { 1, 2, 3 };

    public IEnumerator<int> GetEnumerator()
    {
        foreach (var num in numbers)
            yield return num;
    }

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}
```

112. How do you implement and use async methods in C#?

- **Async methods** are methods that run asynchronously using the async keyword. They return Task or Task<T>, and you use await inside these methods to pause execution until the awaited task completes.

```
public async Task<int> GetDataAsync()
```

```
{
    var data = await SomeAsyncMethod();
    return data;
}
```

113. What is Reflection and what are the use cases for it in C#?

- **Reflection** allows you to inspect the metadata of assemblies, types, and members at runtime. Use cases include:
    - Dynamically invoking methods.
    - Loading assemblies dynamically.
    - Generating code or performing actions based on types at runtime.

```
Type type = typeof(MyClass);
MethodInfo method = type.GetMethod("MyMethod");
method.Invoke(new MyClass(), null);
```

114. What is yield return in C#?

- **yield return** allows you to define an iterator method, which provides elements one at a time. It helps implement deferred execution for collections or sequences.

```
public IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

115. What is the difference between abstract class and interface?

- **Abstract class** can provide method implementations, while **interface** can only declare methods without implementation.
- A class can inherit only one **abstract class**, but it can implement multiple **interfaces**.

```
// Abstract Class
public abstract class Animal
{
    public abstract void Speak();
}

// Interface
public interface IAnimal
{
    void Speak();
}
```

116. Explain the concept of lazy initialization in C#?

- **Lazy initialization** refers to deferring the creation of an object until it is actually needed. In C#, you can use Lazy<T> to implement lazy initialization.

```
Lazy<MyClass> myClass = new Lazy<MyClass>(() => new MyClass());
MyClass obj = myClass.Value; // Object is created only when needed
```

117. What is Dependency Injection and how does it work in C#?

- **Dependency Injection (DI)** is a design pattern that allows you to inject dependencies (objects) into a class rather than hard-coding them within the class. This helps in decoupling and improving testability.

```
public class Car
{
    private readonly IEngine _engine;

    public Car(IEngine engine)
    {
        _engine = engine;
```

```
    }
}


// Usage
IEngine engine = new Engine();
var car = new Car(engine);
```

118. What is Serialization and Deserialization in C#?

- **Serialization** is the process of converting an object into a format (like JSON or XML) for storage or transmission, and **Deserialization** is converting the serialized data back to the original object.

```
// Serialize to JSON
var json = JsonConvert.SerializeObject(myObject);


// Deserialize from JSON
var myObject = JsonConvert.DeserializeObject<MyClass>(json);
```